S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11
Postlab (OPEN ENDED PRACTICAL)

Aim: To implement the Round Robin CPU Scheduling Algorithm and calculate the waiting time and turnaround time for given processes.

Name: Narayani Wadhai

USN:CM24025

Semester / Year:4th/2nd

Academic Session:2025-26

Date of Performance:

Date of Submission:

❖ **Aim:** To implement the Round Robin CPU Scheduling Algorithm and calculate the waiting time and turnaround time for given processes.

❖ **Objectives:**

- To understand the concept of CPU scheduling
- To implement Round Robin algorithm
- To calculate waiting time and turnaround time
- To analyze the performance of time-sharing systems

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

Round Robin is a **preemptive CPU scheduling algorithm** used in time-sharing systems.

- Each process is assigned a fixed time slice called **Time Quantum**
- CPU executes a process for that time
- If process is not completed, it goes to the end of the queue
- This continues until all processes are finished

✓ **Key Points:**

- Fair allocation of CPU
- Avoids starvation
- Suitable for multitasking systems

❖ **Algorithm**

1. Input number of processes
2. Input burst time for each process
3. Input time quantum
4. Initialize remaining burst time
5. Execute processes in cyclic order
6. If remaining time > quantum → subtract quantum
7. Else → finish process
8. Calculate:
    - Turnaround Time = Completion Time
    - Waiting Time = Turnaround Time - Burst Time

❖ CODE:

```
#include <stdio.h>

int main() {
    int n, i, time = 0, remain, tq;
    int wt = 0, tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n];

    printf("Enter burst time for each process:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rt[i] = bt[i]; // remaining time
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    remain = n;

    while(remain != 0) {
        for(i = 0; i < n; i++) {
            if(rt[i] > 0) {
                if(rt[i] <= tq) {
                    time += rt[i];
                    wt += time - bt[i];
                    tat += time;
                    rt[i] = 0;
                    remain--;
                } else {
                    time += tq;
                    rt[i] -= tq;
                }
            }
        }
    }

    printf("\nAverage Waiting Time = %.2f", (float)wt/n);
```

```
    printf("\nAverage Turnaround Time = %.2f\n", (float)tat/n);

    return 0;
}
```

- **Output:**

```
Enter number of processes: 3
Enter burst time for each process:
5 20 25
Enter time quantum: 5

Average Waiting Time = 15.00
Average Turnaround Time = 31.67


...Program finished with exit code 0
Press ENTER to exit console.
```

- **Conclusion**: The Round Robin algorithm ensures fair CPU allocation among processes. It improves system responsiveness and is widely used in time-sharing systems..

- **Discussion Questions:**

  1. What is Round Robin Scheduling?

  2. What is Time Quantum?

  3. What are the advantages of Round Robin?

  4. What are the disadvantages of Round Robin?

- **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/
https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26